

자기소개서

안녕하십니까. 사용자 경험과 기술 구조를 함께 고민하는 프론트엔드 개발자 김희준입니다.

저는 비전공자이지만, 사용자 중심의 관점으로 문제를 정의하고, 필요한 기술을 빠르게 익혀 구현하며, 팀원과 소통해 문제를 해결할 수 있는 개발자입니다.

실내건축디자인에서 익힌 사용자 중심의 관점은 프론트엔드 개발의 밑바탕이 되었습니다

실내건축디자인을 전공하며 공간의 목적을 정의하고, 사용자가 무엇을 보고 어떻게 이동하며 어떤 경험을 하게 될지를 설계했습니다. 이때 익힌 관점은 프론트엔드에서도 기능을 배치하기 전에 사용자의 목적과 행동 흐름을 먼저 생각하는 기준으로 이어졌습니다.

창업 과정에서는 체험단의 반응을 관찰해 상품 구성과 설명 방식을 개선했습니다. 판매 성과는 낮지만, 고객 반응과 판매 정보를 축적해 다음 판단에 활용하는 구조가 없어 사업을 지속하지는 못했습니다.

이 실패를 통해 사용자의 문제를 발견하는 것과 이를 반복 가능한 시스템으로 해결하는 것은 다르다는 사실을 깨달았습니다. 직접 시스템을 만들기 위해 개발을 시작했고, 크래프톤 정글에 참여했습니다.

필요한 기술을 빠르게 익혀 구현합니다

크래프톤 정글에서 자료구조와 알고리즘, 운영체제, 네트워크를 학습하며 서비스가 동작하는 기반을 익혔습니다.

5주 안에 9천만 건의 틱 데이터를 다루는 커스텀 주식 차트 엔진을 구현했습니다. 기존 차트 API에서 지원하지 않는 틱 데이터와 분석 결과를 다양한 형태로 오버레이해야 했기 때문에 TypeScript로 차트를 직접 개발했습니다. 과거 데이터는 REST API로 조회하고, 실시간 데이터는 WebSocket을 통해 **평균 초당 약 1,080건씩** 반영했습니다.

대량의 데이터가 갱신되는 상황에서도 사용자 상호작용이 끊기지 않도록 멀티 레이어 Canvas를 적용했습니다. 정적인 차트와 포인터 UI를 서로 다른 레이어로 분리해, 포인터가 움직일 때 전체 차트를 다시 그리지 않고 필요한 영역만 갱신했습니다.

43종의 기능 패널에는 공통 프레임을 적용하고, TypeScript Registry에서 크기와 배치, 우선순위와 동작 기준을 관리했습니다. 기능이 늘어나도 사용자가 같은 방식으로 서비스를 이해하고 조작할 수 있도록 코드 구조와 사용 경험을 함께 고려했습니다.

또한 React를 사용하는 데서 멈추지 않고 **Virtual DOM과 Diff/Patch, Hooks, batching을 직접 구현했습니다.** 상태가 저장되고 변경된 내용이 렌더링으로 이어지는 과정을 코드로 확인했습니다. 동작하지 않는 부분은 상태의 저장 위치와 갱신 순서를 추적하며 원인을 찾았고, 이를 통해 복잡한 렌더링 문제를 구조적으로 분석할 수 있는 기반을 쌓았습니다.

소통을 통해 문제를 빠르게 파악하고 해결합니다

자동차 부품 제조 현장에서 생산 라인장으로 근무하며 20명의 현장 인력과 엔지니어, 외주업체, 납품업체 사이의 업무를 조율했습니다. 문제가 발생하면 혼자 판단하기보다 관련 담당자에게 상황을 확인해 원인이 발생한 지점을 찾고, 생산 일정과 품질 기준에 맞춰 대응 방안을 정했습니다. 필요한 정보를 빠르게 모으고 각 담당자의 업무를 연결하며 생산 차질을 줄였습니다.

개발 과정에서도 문제의 원인이 화면과 API, 데이터 등 여러 영역에 걸쳐 있을 수 있습니다. 문제가 생겼을 때 혼자 오래 붙잡고 있기보다 재현 조건과 확인한 내용을 팀원에게 정확히 공유하고, 필요한 담당자와 함께 원인을 찾아 빠르게 해결하겠습니다.

특히 '나만의 무기' 프로젝트를 진행하며 프론트엔드를 진로로 결정했습니다. 처음에는 UI를 완성된 기능을 화면에 배치하는 과정이라고 생각했습니다. 그러나 같은 데이터와 AI 기능도 무엇을 먼저 보여주고 다음 행동을 어떻게 유도하느냐에 따라 단순한 정보 조회 서비스가 될 수도, 사용자의 판단을 돕는 투자 작업 공간이 될 수도 있었습니다.

이 과정에서 **프론트엔드는 데이터의 상태 변화와 사용자 상호작용을 연결해 프로젝트의 콘셉트와 역할을 사용자 경험으로 구체화하는 영역**이라는 확신을 얻었습니다. 프론트엔드는 서비스 개발의 마지막 단계가 아니라, 기획과 데이터, 백엔드가 만든 결과가 사용자에게 어떤 의미로 전달될지를 결정하는 영역이었습니다.

업무에서는 요구사항을 곧바로 화면에 옮기기보다 서비스가 해결하려는 문제와 사용자의 목적, 데이터 상태를 먼저 정리하겠습니다. 문제가 발생하면 재현 조건과 원인을 확인하고, 해결 과정은 코드와 문서에 남겨 같은 문제가 반복되지 않도록 하겠습니다.

문제가 생겼을 때 포기하지 않고 원인을 끝까지 찾아, 사용자에게는 더 나은 경험을 제공하고 팀에는 다시 활용할 수 있는 해결책을 남기는 프론트엔드 개발자가 되겠습니다.

감사합니다.